

Tests, validation et corrections

MediPlan — projet intégrateur 030747 · Collège La Cité · août 2026 Équipe : Souleymane DIALLO · Zakaria Lahouiri · Larbi Saib

1. Ce que nous testons, et pourquoi

Nous n'avons pas cherché à couvrir tout le code. À trois, sur un semestre, une couverture uniforme aurait consommé le temps du développement sans rien garantir de plus. Nous avons choisi de tester **ce qui casse silencieusement** :

Priorité	Ce que ça recouvre	Pourquoi
1	Les règles métier	Une transition de statut invalide ou un créneau réservé deux fois corrompt des données, sans message d'erreur
2	La sécurité	Un défaut de contrôle d'accès ne se voit jamais à l'usage : tout fonctionne, simplement trop de monde y a accès
3	Le comportement des écrans	Un formulaire qui accepte une saisie invalide produit une erreur serveur incompréhensible
hors périmètre	L'apparence	Un test qui vérifie une couleur casse à chaque retouche et n'attrape aucun défaut réel

Ce choix a une conséquence assumée : **nous n'avons pas de tests de bout en bout automatisés**. Le parcours complet est validé à la main, avant chaque démonstration, selon le scénario écrit dans [SCENARIO-DEMO-Finale.md](../presentation/SCENARIO-DEMO-Finale.md).

2. Ce qui tourne

203 tests automatisés, tous verts au 12 août 2026.

Backend — 70 tests, 11 suites

Suite	Ce qu'elle vérifie
auth.service.spec	Inscription (dont choix de la clinique, validé en base), connexion, hachage, verrouillage après

	échecs, réinitialisation
<code>password-policy.validator.spec</code>	Politique de mot de passe
<code>roles.guard.spec</code>	Le garde de rôle laisse passer ou refuse selon le rôle porté par le jeton
<code>users.service.spec</code> · <code>users.controller.spec</code>	Création du patient léger, périmètre de clinique, profil courant
<code>availability.service.spec</code> · <code>availability.controller.spec</code>	Plages datées, génération et publication immédiate des créneaux, bornes invalides
<code>appointments.controller.spec</code>	Routes, gardes et validation des entrées
<code>appointments.service.spec</code>	Émission des notifications sur les trois événements du cycle de vie ; réservation par le patient : identité issue du jeton, anti-IDOR, créneau passé, conflit
<code>notifications.service.spec</code>	Destinataires, contenu, marquage comme lu
<code>app.controller.spec</code>	Sonde de disponibilité

Frontend — 133 tests, 28 suites

Regroupés en trois familles :

- **Noyau** — façade d'authentification, intercepteurs (jeton porteur, gestion du 401), garde de rôle, service de thème, coquille applicative et navigation par rôle ;
- **Écrans** — connexion, inscription, mot de passe oublié, réinitialisation, liste des utilisateurs, tableau de bord ;
- **Services et composants partagés** — services HTTP par domaine, messages d'erreur, avatar, badge de rôle, carte de statistique, état vide.

Ce qui n'est pas couvert

- Aucun test de bout en bout automatisé — parcours validé manuellement ;
- Aucun test de charge ;
- Les écrans **Statistiques**, **Flux du jour**, **Mes rendez-vous** et le formulaire d'export CSV n'ont pas de test de composant : leurs services HTTP sont testés, leurs gabarits non ;
- Le jeu de démonstration n'a pas de test : il est vérifié en le rejouant et en interrogeant la base.

3. La barrière qualité : la CI

Configurée dans [.github/workflows/ci.yml](../.github/workflows/ci.yml), déclenchée à chaque push et sur chaque pull request.

Étape	Bloquante ?
Compilation du backend et du frontend	oui
Les 203 tests	oui
Validation des gabarits d'infrastructure Bicep	oui
Lint et formatage	non — rapportés seulement

Une pull request dont la CI est rouge n'est pas fusionnée. Les 29 pull requests du projet sont passées par là.

Pourquoi le lint ne bloque pas

C'est un choix, pas un oubli. Le dépôt traîne une dette de formatage : **233 fichiers** n'ont jamais été passés au formateur et le frontend porte **9 avertissements** d'accessibilité clavier. Rendre le lint bloquant aujourd'hui donnerait une CI **rouge en permanence** — et une CI toujours rouge, plus personne ne la regarde. Nous avons préféré qu'elle reste crédible sur ce qui compte : la compilation et les tests.

La dette est identifiée, chiffrée, et inscrite dans nos pistes d'amélioration.

4. La vérification qui compte le plus : la double réservation

C'est notre premier objectif produit. Il mérite mieux qu'un test unitaire.

Le mécanisme

Un **index unique partiel** en base, posé par la migration 1781674897615-AddAppointmentReceptionBooking :

```
CREATE UNIQUE INDEX "uq_appointment_active_slot" ...
```

Il est **partiel** : il exclut les rendez-vous annulés. C'est ce qui permet à un créneau annulé de redevenir reservable, sans code supplémentaire.

Côté service, la ligne du créneau est verrouillée (pessimistic_write) pendant la transaction, et une violation d'unicité est traduite en **HTTP 409**.

La vérification réelle — 10 août 2026, sur l'application déployée

Deux requêtes de réservation lancées **strictement en parallèle** sur le même créneau libre :

```
req A & req B & wait
```

Résultat :

```
requête A -> HTTP 409  {"statusCode":409,"error":"Conflict",
                        "message":"Ce créneau est déjà réservé."}
requête B -> HTTP 201  {"id":"cd669bbb-...", "slotId":"268c5321-...", ...}
```

Une seule réservation passe. La seconde est refusée proprement, avec un message destiné à l'utilisateur — pas une erreur serveur.

Honnêteté sur ce point : cette vérification est manuelle et reproductible, pas automatisée. Nos tests backend sont des tests unitaires avec dépendances simulées : ils ne peuvent pas prouver un comportement qui appartient au moteur de base de données. Un test automatisé exigerait une vraie base dans la chaîne d'intégration. C'est inscrit dans nos pistes d'amélioration.

La même vérification sur le second canal — 11 août 2026

Depuis l'ouverture de la réservation en libre-service (MEDIPLAN-21), un créneau peut être pris de **deux façons** : par la réception, ou par le patient. La question devient donc « la garantie tient-elle *entre* les deux canaux ? ».

Elle tient, parce qu'elle n'est pas dans le code applicatif. Les deux chemins passent par la même transaction, le même verrou de ligne et le même index. Rejoué en local sur deux réservations **patient** simultanées :

```
requête A -> HTTP 201
requête B -> HTTP 409  « Ce créneau est déjà réservé. »
```

Et de bout en bout : un patient réserve 9 h 00 en ligne, la réception ouvre la même plage — **9 h 00 n'est plus dans sa liste**. C'est cette scène qui ouvre la démonstration du 13 août.

5. Bogues rencontrés et corrigés

Sept défauts nous ont réellement coûté du temps. Quatre ont un point commun : **ils n'existaient pas sur nos postes** ; le dernier n'existait que pour un utilisateur que nous n'avions pas encore.

5.1 — Le 404 sur toute l'API, en production seulement

Symptôme. Une fois l'application en ligne, tous les appels /api renvoyaient

1. En local, tout fonctionnait.

Diagnostic. Nous avons d'abord cherché dans le code, puis dans les routes, puis dans la configuration. La cause était dans le proxy : il transmettait l'en-tête Host demandé par le navigateur au lieu de celui de la destination. **Azure Container Apps route selon cet en-tête** ; Docker Compose, en local, ne route pas du tout de cette façon.

Correction. Transmettre l'hôte de destination (PR [#17](https://github.com/soultaka19/mediplan/pull/17)).

Ce que nous en retenons. Un environnement de développement qui fonctionne ne prouve rien sur la production. Les deux ne diffèrent pas seulement par leurs données.

5.2 — Le conteneur qui refusait de démarrer

Symptôme. L'image se construisait, le conteneur mourait au lancement sans message exploitable.

Diagnostic. Nos postes sont sous Windows avec conversion automatique des fins de ligne. Le script de démarrage était récupéré avec des fins de ligne CRLF, que l'interpréteur du conteneur ne sait pas lire.

Correction. Forcer les fins de ligne LF sur les scripts shell au niveau du dépôt, via `.gitattributes`.

5.3 — L'application blanche au démarrage

Symptôme. Erreur Angular NG0200 au chargement, écran vide.

Diagnostic. Une dépendance circulaire dans la façade d'authentification, déclenchée par le rafraîchissement du profil au démarrage.

Correction. Casser le cycle en différant la récupération du profil (commit 81f8e71).

5.4 — Une clinique ouverte à 4 h 30 du matin

Symptôme. Après avoir peuplé la base en production, le flux du jour affichait des rendez-vous de 4 h 30 à 10 h.

Diagnostic. Le jeu de démonstration posait ses horaires avec `setHours()`, qui travaille dans le fuseau **du serveur**, alors que l'application affiche en `America/Toronto`. Sur nos postes, réglés à l'heure de l'Est, les deux coïncident — c'est pourquoi le défaut a survécu tout le projet. Le conteneur tourne en UTC : décalage de quatre heures.

Correction. Viser explicitement l'heure murale de la clinique, décalage du fuseau calculé à l'instant visé (PR [#24](https://github.com/soultaka19/mediplan/pull/24)).

Vérification. Le seed rejoué avec `TZ=UTC` et en heure de l'Est donne désormais le même résultat : journée de 8 h 30 à 16 h 30.

5.5 — Le bogue de méthode : la dérive d'intégration

Ce n'est pas un défaut de code, et c'est celui qui nous a le plus appris.

Symptôme. Cinq tickets marqués « Terminé » dans Jira, absents du produit.

Diagnostic. Nous travaillions chacun sur nos branches sans les fusionner assez souvent. Certaines avaient pris **jusqu'à 56 commits de retard**, dont une refonte complète de l'interface. Notre définition de « terminé » était trop souple : elle disait « codé » là où elle aurait dû dire « fusionné ».

Correction. Trois fonctionnalités réintégrées en trois pull requests revues (PR [#19](https://github.com/soultaka19/mediplan/pull/19), [#20](https://github.com/soultaka19/mediplan/pull/20), [#21](https://github.com/soultaka19/mediplan/pull/21)), conflits résolus en conservant les deux apports. Les tickets dont le code n'était pas fusionné sont repassés « À faire », avec la raison écrite dans Jira.

Notre nouvelle règle. Terminé = fusionné dans main, CI verte.

5.6 — Le créneau qui n'existait pas encore

Symptôme. Un patient ne voyait aucun créneau sur une plage que la réception venait de publier. Depuis le comptoir, tout paraissait normal.

Diagnostic. Les créneaux n'étaient créés en base qu'au premier appel de POST `/availabilities/:id/slots` — appel déclenché par le **dialogue de réservation de la réception**. Tant qu'ils n'existent pas, ils n'ont pas d'identifiant : il n'y a rien à réserver.

Le défaut était invisible depuis la réception, puisque le geste qui aurait pu le révéler était précisément celui qui le corrigeait au passage. Il n'est apparu qu'avec un second acteur, le patient, qui ne déclenche jamais cette création.

Correction. Publier les créneaux **dès la création de la plage**, via une insertion idempotente qui n'écrase jamais un créneau déjà réservé (PR [#26](https://github.com/soultaka19/mediplan/pull/26)).

Ce que nous en retenons. Un état construit « au premier accès » n'est pas construit : il est construit *pour celui qui accède le premier*. Tant qu'un seul type d'utilisateur passait par là, personne ne pouvait le voir.

5.7 — Le créneau qui survivait à sa plage

Symptôme. Supprimer une disponibilité renvoyait 204, mais ses créneaux restaient proposés à la réservation — pour une matinée qui n'existait plus.

Diagnostic. Aucune clé étrangère ne rattache un créneau à la plage qui l'a publié. Le défaut préexistait, mais restait hors d'atteinte tant que les créneaux n'étaient créés qu'à l'ouverture du dialogue de la réception. Depuis le correctif précédent (§5.6), ils existent toujours — donc toute suppression laissait des orphelins, réservables par un patient.

Correction. La suppression retire les créneaux **libres** de la plage. Si l'un d'eux est déjà réservé, elle est **refusée en 409** : effacer la matinée d'un médecin ne doit pas faire disparaître en silence les rendez-vous de ses patients (PR [#28](https://github.com/soultaka19/mediplan/pull/28)).

Ce que nous en retenons. Corriger un défaut peut en rendre un autre atteignable. Celui-ci dormait depuis des semaines ; c'est le correctif de la veille qui l'a mis sur le chemin des utilisateurs.

6. Stabilité de la solution

Constaté sur l'environnement déployé, le 11 août 2026 :

Point	Résultat
Erreurs dans la console du navigateur, parcours complet	aucune
Connexion, réservation (réception et patient), cycle de vie, annulation, export, statistiques	tous fonctionnels
Réservations simultanées sur le même	1 acceptée, 1 refusée en 409

créneau, sur les deux canaux	
Migrations rejouées au démarrage du conteneur	idempotentes — un réveil ne rejoue rien
Dernière exécution de la CI sur main	verte

Limite connue de stabilité : les conteneurs s'arrêtent quand personne ne les utilise (*scale-to-zero*), ce qui met l'hébergement à environ 0 \$/mois. Le premier accès après une période d'inactivité prend **10 à 15 secondes**. Ce n'est pas une panne, mais c'est visible, et il faut réveiller l'application avant une démonstration.

7. Ce que nous ferions ensuite

Par ordre de valeur décroissante :

2. **Une base de données dans la chaîne d'intégration**, pour transformer la vérification manuelle de concurrence (§4) en test automatisé — c'est notre plus gros manque de couverture ;
3. **Des tests de bout en bout** sur le parcours complet, pour ne plus dépendre d'une validation manuelle avant chaque démonstration ;
4. **Résorber la dette de formatage**, puis rendre le lint bloquant ;
5. **Des tests de composant** sur les écrans Statistiques, Flux du jour et Mes rendez-vous ;
6. **L'annulation par le patient**, avec sa règle de délai minimum (UC-07) — nous avons préféré ne pas livrer l'une sans l'autre ;
7. Des tests de charge, si le produit devait servir plusieurs cliniques.